



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Jesús Alexis Pérez León

N° de Cuenta: 314207850

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 23-abril-2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1.- Hacer que su carro recorra la pista **Adecuadamente (curvas, inclinación en rampa, faro frontal ilumina la pista)** desde el inicio hasta el punto extremo y ahí se detenga. Se puede repetir la animación si se presiona una tecla designada.

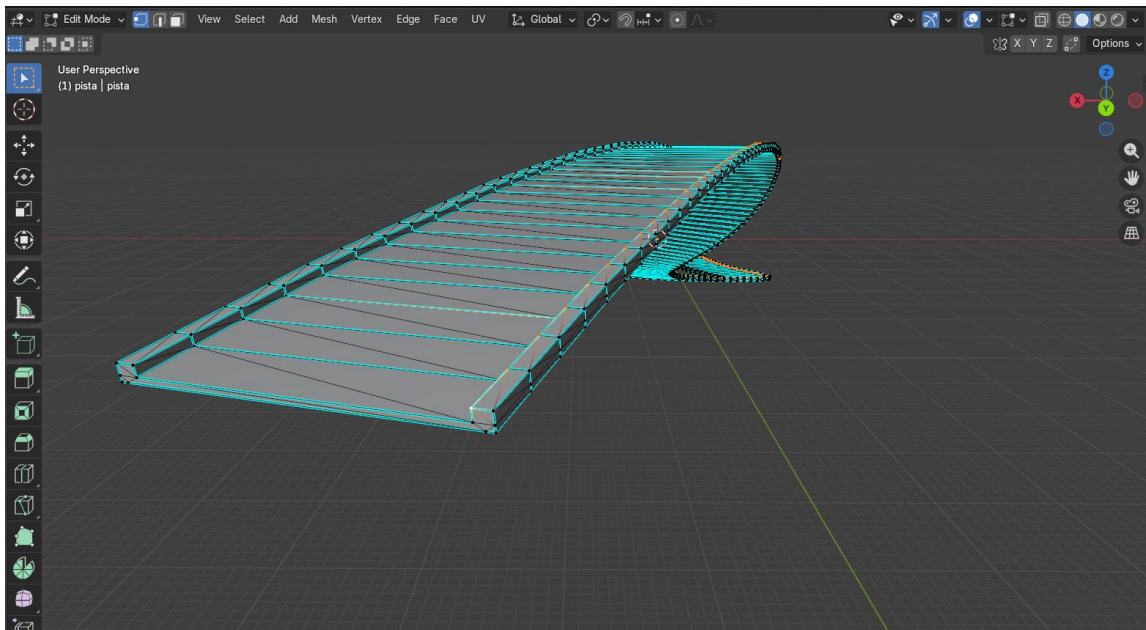
2.-Separar de la Nave el otra ala y las dos hélices, Hacer que sobrevuele la pista **adecuadamente(jerarquía, aleteo, giro de hélices, faro ilumina la pista)** pero en sentido contrario, inicia en el punto extremo y aterriza junto al punto inicial. Esta animación no se puede repetir.

Extra 5 puntos: Su dado de 8 caras cae girando sobre el piso y muestra un número random **adecuadamente** hacia arriba, se repite la tirada al presionar una tecla.

1)

Para la animación del vehículo primero obtuve los puntos de la superficie de la pista.

Importe la pista en blender y seleccione los vértices de las orillas que van a lo largo de la pista.



Y mediante el siguiente script en Python blender me exporto los puntos-vertices (x,y,z) en un txt y estos los agregue en mi main.cpp

```
File Edit Render Window Help Layout Modeling Sculpting UV Editing Texture Paint Shading Animation
View Text Edit Select Format Templates Text
1 import bpy
2 import os
3
4 # Obtener el objeto activo y su malla
5 obj = bpy.context.active_object
6 mesh = obj.data
7
8 # Cambiar a modo objeto para que Blender actualice la selección de vértices
9 bpy.ops.object.mode_set(mode='OBJECT')
10
11 # Definir la ruta (Escritorio en este caso)
12 # Puedes cambiar 'coordenadas_pista.txt' por el nombre que prefieras
13 desktop_path = os.path.join(os.path.expanduser("~"), "Desktop", "coordenadas_pista.txt")
14
15 with open(desktop_path, 'w') as f:
16     f.write("X, Y, Z\n") # Encabezado
17
18     for v in mesh.vertices:
19         if v.select:
20             # Multiplicamos por la matriz del mundo para obtener coordenadas globales
21             # Esto es vital si moviste, rotaste o escalaste el objeto en Object Mode
22             global_coord = obj.matrix_world @ v.co
23             line = f"{global_coord.x:.4f}, {global_coord.y:.4f}, {global_coord.z:.4f}\n"
24             f.write(line)
25
26 print(f"Exportación finalizada en: {desktop_path}")
27
28 # Volver a modo edición por comodidad
29 bpy.ops.object.mode_set(mode='EDIT')|
```

```
//Puntos de la superficie de la pista (obtenidos en Blender)
std::vector<glm::vec3> puntosPista = {
    //glm::vec3(-5.3678f, -1.6908f, 0.6023f),
    //glm::vec3(-5.3678f, -0.8373f, 0.6023f),
    glm::vec3(-5.3737f, 1.5927f, 0.5995f),
    glm::vec3(-5.3845f, 3.2439f, 0.5945f),
    glm::vec3(-5.3971f, 4.8881f, 0.5893f),
    glm::vec3(-5.4151f, 6.5241f, 0.5843f),
    glm::vec3(-5.4411f, 8.1549f, 0.5798f),
    glm::vec3(-5.4763f, 9.7823f, 0.5756f),
    glm::vec3(-5.5233f, 11.4062f, 0.5720f),
    glm::vec3(-5.5818f, 13.0272f, 0.5689f),
    glm::vec3(-5.6531f, 14.6447f, 0.5665f),
    glm::vec3(-5.7385f, 16.2586f, 0.5647f),
    glm::vec3(-5.8390f, 17.8680f, 0.5637f),
    glm::vec3(-5.9559f, 19.4723f, 0.5634f),
    glm::vec3(-6.0904f, 21.0707f, 0.5640f),
    glm::vec3(-6.2439f, 22.6621f, 0.5655f),
    glm::vec3(-6.4210f, 24.2439f, 0.5682f),
    glm::vec3(-6.6242f, 25.8146f, 0.5722f),
    glm::vec3(-6.8541f, 27.3742f, 0.5775f),
    glm::vec3(-7.1127f, 28.9224f, 0.5843f),
    glm::vec3(-7.4022f, 30.4584f, 0.5927f),
    glm::vec3(-7.7246f, 31.9815f, 0.6031f),
    glm::vec3(-8.0822f, 33.4907f, 0.6156f),
    glm::vec3(-8.4791f, 34.9843f, 0.6308f),
    glm::vec3(-8.9175f, 36.4614f, 0.6493f),
    glm::vec3(-9.3954f, 37.9231f, 0.6712f),
    glm::vec3(-9.9138f, 39.3702f, 0.6974f),
    glm::vec3(-10.4732f, 40.8049f, 0.7286f),
}
```

Lo anterior me apoye de la IA GEMINI:

tengo un OBJ de una autopista, necesito obtener puntos de la superficie de la pista para que un automóvil los recorra, lo puedo hacer en blender?

◆ ¡Claro que sí! Blender es excelente para esto. Dependiendo de qué tan "automático" o manual quieras el proceso, tienes tres caminos principales.

Aquí tienes las mejores formas de extraer esos puntos:

1. El método de "Vértices a Curva" (Preciso y Manual)

Si tu autopista tiene una topología limpia (es decir, una línea de polígonos que recorre el centro o los bordes), este es el método más fiel.

1. **Entra en Edit Mode** (`T`): Selecciona un "loop" de bordes o una fila de vértices que represente la trayectoria que quieres (por ejemplo, la línea central de la pista).
2. **Duplica la selección**: Presiona `Shift + D` y luego `P` > **Selection** para separar esos vértices en un objeto nuevo.
3. **Convierte a Curva**: Sal a Object Mode, selecciona el nuevo objeto de cables y ve a **Object > Convert > Curve**.
4. **Exportar puntos**: Ahora que es una curva, puedes remuestrearla (en las propiedades de la curva, baja la resolución) y usar un script de Python simple para obtener las coordenadas (x, y, z) .

2. Geometry Nodes (El más potente y flexible)

Si quieres generar puntos distribuidos uniformemente a lo largo de la superficie sin volverte loco, **Geometry Nodes** es tu mejor amigo.

- **Paso A:** Crea un plano largo o una curva simple que siga la forma general de la pista.
- **Paso B:** Usa el nodo **Raycast**. Puedes proyectar puntos desde arriba hacia tu objeto OBJ (la autopista). El nodo detectará dónde "chocan" los rayos con la superficie y te dará la posición exacta en el eje Z .
- **Paso C:** Usa **Mesh to Points** para visualizar dónde estarían los puntos de apoyo de las ruedas.

3. Scripting (Para programadores)

Si necesitas los puntos para meterlos directamente en un motor de juegos (como Unity o Unreal) o un simulador, puedes extraer la posición de los vértices directamente con la consola de Python de Blender:

Después mediante una función específica para el auto, calculamos el `translate` (avance de frente del auto) y `rotate's` (giro e inclinación) a partir de ir iterando los puntos de la pista de dos en dos, lo que permite un avance fluido y coherente sobre la pista. Adicional y para mejor ajuste, se implementaron offsets para centrar y ajustar la altura del vehículo sobre la pista y en función a la velocidad del auto se aplicó una rotación a las llantas proporcional a la misma. La función se llama en el loop principal en función a `deltaTime` para tener una tasa de cuadros constante.

```

// Datos para manejo de puntos de la pista y movimiento del auto y nave
int i = 0;           // Índice del punto actual
float t = 0.0f;      // Progreso entre punto i e i+1 (0.0 a 1.0)
float velocidad = 0.5f;
float distancia;

// Variables para el Translate y Rotate del auto
glm::vec3 posAuto;
float angleY = 0.0f; // YAW (giro horizontal)
float angleX = 0.0f; // PITCH (si sube o baja una rampa)
float angleZ = 0.0f; // ROLL (giro lateral)

//variables para el moviminto del dado
bool estaAnimando = false;
float tiempoInicioAnim = 0.0f;
float anguloX = 0.0f, anguloY = 0.0f, anguloZ = 0.0f;

```

```

void movimientoAuto(float deltaTime) {
    if (puntosPista.empty() || i >= puntosPista.size() - 1) return;
    //Definir puntos de origen y destino
    glm::vec3 p0 = puntosPista[i];
    glm::vec3 p1 = puntosPista[i + 1];
    // 3. Control de progreso (t)
    // Calculamos la distancia real entre los puntos de Blender
    distancia = glm::distance(p0, p1);
    t += (velocidad * deltaTime) / distancia;
    // Si llegamos al destino, pasamos al siguiente tramo
    if (t >= 1.0f) {
        t = 0.0f;
        i++;
        return;
    }

    // Posición interpolada
    glm::vec3 posInterp = p0 + t * (p1 - p0);
    posAuto.x = posInterp.x;
    posAuto.y = posInterp.z; // La Z de Blender es la altura en OpenGL
    posAuto.z = posInterp.y; // La Y de Blender es la profundidad en OpenGL
    rotllanta -= (velocidad * 30.0f) * deltaTime;

    //CÁLCULO DE DIRECCIÓN (Vectores)
    glm::vec3 direccion;
    direccion.x = p1.x - p0.x;
    direccion.y = p1.z - p0.z; // Diferencia de altura real
    direccion.z = p1.y - p0.y; // Diferencia de profundidad real
    direccion = glm::normalize(direccion);

    //CÁLCULO DE ÁNGULOS (En grados para glm::rotate)
    // angleY: Giro horizontal (Yaw)
    angleY = glm::degrees(atan2(direccion.x, direccion.z));
    // angleX: Inclinación de rampa (Pitch)
    angleX = glm::degrees(-asin(direccion.y));
}

```

Para evitar ajustes innecesarios relacionados a la posición de la pista, esta se vuelve el nodo padre de la jerarquía con el auto y del auto el cofre y las llantas, de esta forma a donde se coloque la pista el auto debe seguir su ruta de manera coherente a la pista. No implemente peralte, pero para un mejor recorrido natural o real se hubiera requerido el conjunto de puntos-vertices de la otra orilla de la pista y así se obtendría un buen resultado de inclinación.

Por último agregue la tecla "X" para reiniciar la posición inicial del vehículo sobre la pista, esta funciona en cualquier momento del recorrido del auto.

```
////////////////////////////////////
//PISTA
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, -2.1f, 0.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));

glm::mat4 modelPista = model; // Base para el auto

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelPista));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
Pista_M.RenderModel();

////////////////////////////////////
//AUTO
model = modelPista;
model = glm::translate(model, glm::vec3(posAuto.x + desplazamientoCentro,
    posAuto.y + alturaSobrePista,
    -posAuto.z));

//Rotación de trayectoria
model = glm::rotate(model, glm::radians(-angleY), glm::vec3(0.0f, 1.0f, 0.0f));
// Ajuste de orientación local
model = glm::rotate(model, -180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
//Rotación de pendiente
model = glm::rotate(model, glm::radians(angleX), glm::vec3(1.0f, 0.0f, 0.0f));
//model = glm::rotate(model, glm::radians(angleZ), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(4.0f, 4.0f, 4.0f));
modelaux = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
auto_M.RenderModel();
```



```

// Faro del auto (Spotlight)
glm::vec3 faroPos = glm::vec3(modelaux * glm::vec4(-0.4f, 0.0f, 1.35f, 1.0f));
glm::vec3 faroDir = glm::normalize(glm::vec3(modelaux * glm::vec4(0.0f, 0.0f, 1.0f, 0.0f)));
spotLights[1].setFlash(faroPos, faroDir);

// COFRE
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.19f, 7.38f));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cofre_M.RenderModel();

//LLANTAS
// Delantera Izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(-1.2f, -0.16f, 0.86f));
model = glm::rotate(model, -(rotllanta)*toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();

// Trasera Izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(-1.2f, -0.16f, -0.9f));
model = glm::rotate(model, -(rotllanta)*toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();

```

```

// Delantera Derecha
model = modelaux;
model = glm::translate(model, glm::vec3(1.2f, -0.16f, 0.86f));
model = glm::rotate(model, -180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, (rotllanta)*toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();

// Trasera Derecha
model = modelaux;
model = glm::translate(model, glm::vec3(1.2f, -0.16f, -0.9f));
model = glm::rotate(model, -180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, (rotllanta)*toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Llanta_M.RenderModel();

```

```

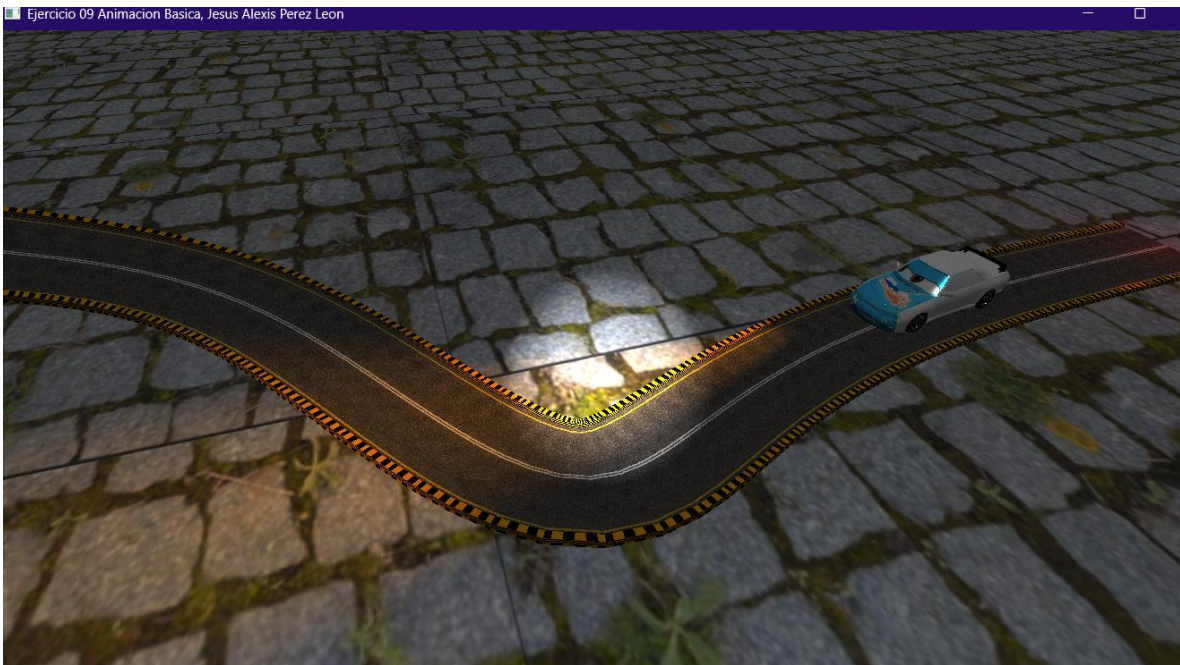
//TECLA DE RESET "X"
if (key == GLFW_KEY_X && action == GLFW_PRESS)
{
    theWindow->resetAuto = true;
}

```

```
// Resetear el auto a su posición inicial si se ha presionado la tecla "X"
if (mainWindow.getResetAuto()) {
    // 1. Reiniciamos variables de la pista
    i = 0;
    t = 0.0f;

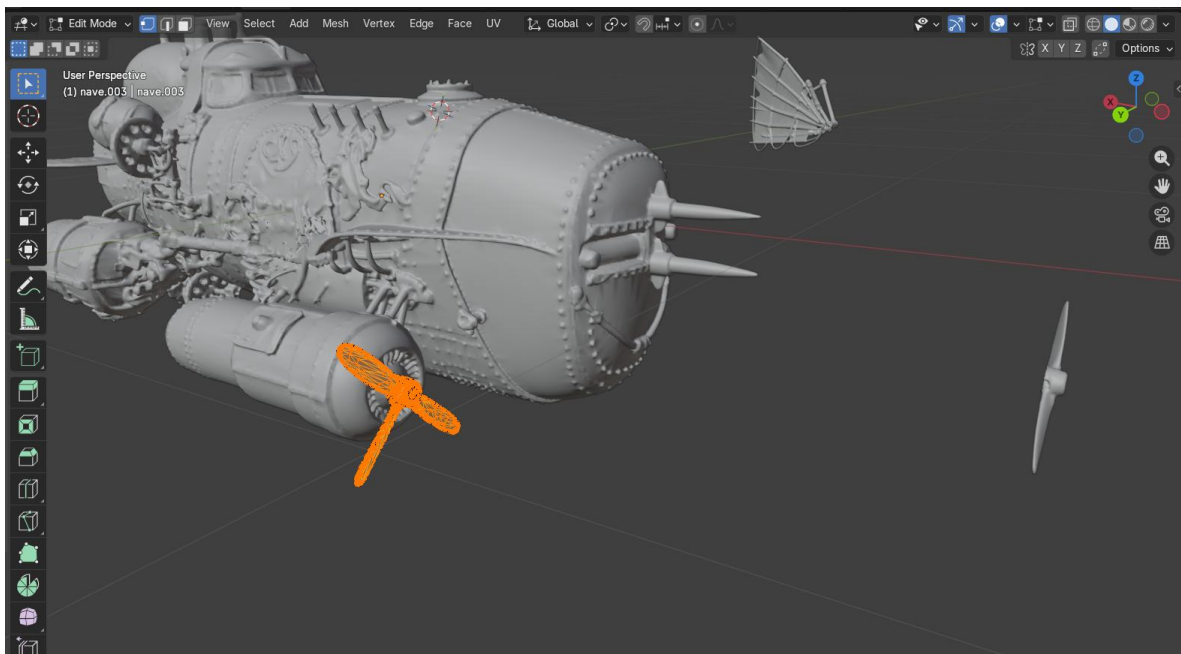
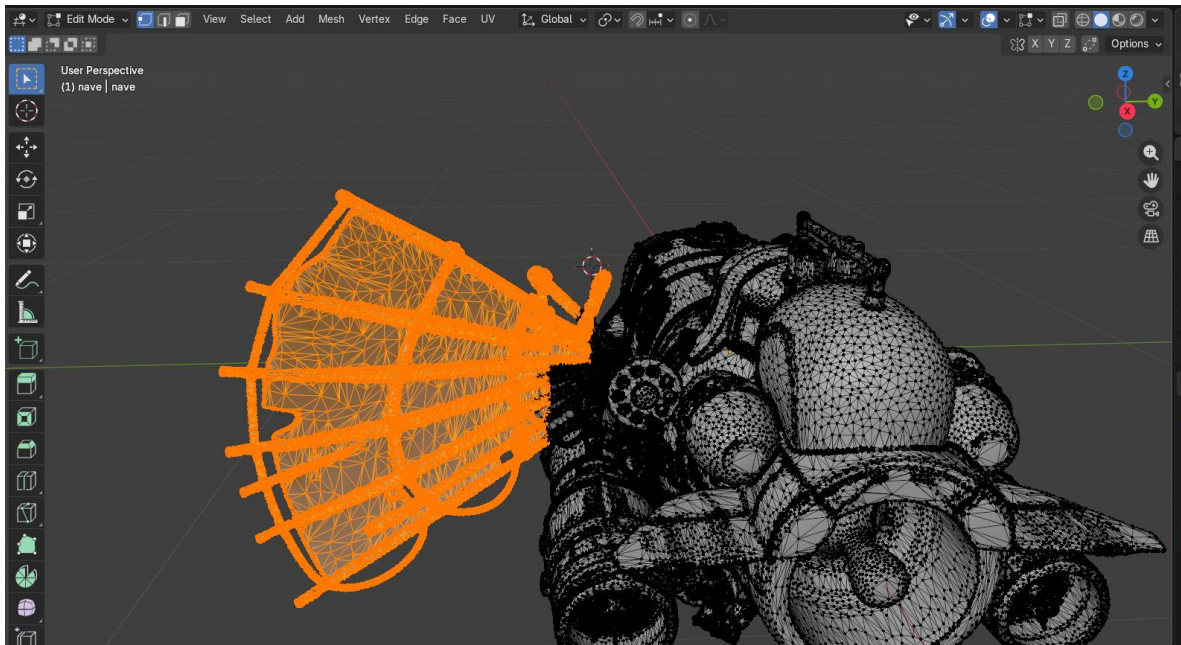
    // 2. Reiniciamos rotaciones
    angleZ = 0.0f;
    rotllanta = 0.0f;

    // 3. APAGAMOS la bandera para que no se quede loopado
    mainWindow.getSetResetAuto(false);
}
```



2)

Pimero separe las alas y hélices del cuerpo de la nave para instanciarlas jerárquicamente en OpenGL y además que tengan su propio movimiento.



Para la nave implemente una función única para esta. Ya que su recorrido no se repite, es en sentido contrario al auto (de fin a inicio) y se debe posicionar (aterrizar) en un lugar específico.

La lógica es exactamente la misma solo que el recorrido o iteración de la pista es de fin a inicio. Toda la animación se realiza mediante tres estados. El primero es el recorrido de la pista. El segundo estado es cuando llega al ultimo punto de la pista se aparta a la izquierda lo que es el comienzo de aterrizar la nave. Y el tercer estado es el descenso de la nave al piso. Durante todo momento las alas y hélices de la nave están en movimiento y es proporcional a la velocidad de la nave.

```

void movimientoNave(float deltaTime) {

    // ESTADO 0: SIGUIENDO LA RUTA
    if (estadoNave == 0) {
        if (puntosPista.size() < 2 || iNave < 0) {
            estadoNave = 1; // Cuando se acaban los puntos, pasamos a maniobra lateral
            return;
        }

        glm::vec3 pOrigen = puntosPista[iNave + 1];
        glm::vec3 pDestino = puntosPista[iNave];

        float dist = glm::distance(pOrigen, pDestino);

        // Evitamos división por cero para evitar saltos/parpadeos
        if (dist > 0.0001f) {
            tNave += (velocidadNave * deltaTime) / dist;
        }
        else {
            tNave = 1.0f;
        }

        if (tNave >= 1.0f) {
            tNave = 0.0f;
            iNave--;
            return;
        }

        glm::vec3 posInterp = pOrigen + tNave * (pDestino - pOrigen);
        posNave.x = posInterp.x;
        posNave.y = posInterp.z + 5.0f;
        posNave.z = posInterp.y;

        // Dirección y Ángulos (Para que no parpadee, solo calculamos si hay movimiento)
        glm::vec3 dir = pDestino - pOrigen;
        if (glm::length(dir) > 0.0001f) {
            dir = glm::normalize(glm::vec3(dir.x, dir.z, dir.y));
            angleYNave = glm::degrees(atan2(dir.x, dir.z));
            angleZNave = glm::degrees(-asin(dir.y));
        }
    }
}

```

```

// ESTADO 1: MÓVERSE A UN LADO (IZQUIERDA/DERECHA)
else if (estadoNave == 1) {
    tManiobra += deltaTime * (velocidadNave / 10.0f); // Velocidad de la maniobra lateral
    desplazamientoLateralFinal = glm::mix(0.0f, 15.0f, glm::min(tManiobra, 1.0f));
    if (tManiobra >= 1.0f) {
        tManiobra = 0.0f;
        estadoNave = 2; // Siguiente paso: bajar
    }
}

// ESTADO 2: BAJAR AL PISO
else if (estadoNave == 2) {
    tManiobra += deltaTime * (velocidadNave / 10.0f); // Velocidad de descenso
    // Bajamos la altura de 5.0 a 0.0 (o lo que midan tus llantas)
    alturaVueloFinal = glm::mix(5.0f, 1.0f, glm::min(tManiobra, 1.0f));

    if (tManiobra >= 1.0f) {
        estadoNave = 3; // Nave aterrizada completamente
    }
}

//Movimiento de las alas, siempre aleteando
rotAla += (velocidadNave * 30.0f) * deltaTime * dirAla;
// Si llega a 35 grados, empieza a CERRARSE EL ALA. Si llega a -45, empieza a ABRIRSE EL ALA.
if (rotAla > 35.0f) {
    rotAla = 35.0f;
    dirAla = -1.0f;
}
else if (rotAla < -45.0f) {
    rotAla = -45.0f;
    dirAla = 1.0f;
}
rothelice += (velocidadNave * 50.0f) * deltaTime;
}

```

Instancia de la nave

```

////////////////////////////////////
//NAVE
model = modelPista;
model = glm::translate(model, glm::vec3(posNave.x + desplazamientoCentroNave + desplazamientoLateralFinal,
    posNave.y + alturaVueloFinal,
    -posNave.z));
model = glm::rotate(model, glm::radians(-angleYNave), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(angleZNave), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(15.0f, 15.0f, 15.0f));
modelauxNave = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Nave_M.RenderModel();

// luz frontal de la nave (Spotlight)
glm::vec3 navePos = glm::vec3(modelauxNave * glm::vec4(-0.5f, -0.1f, 0.0f, 1.0f));
glm::vec3 naveDir = glm::normalize(glm::vec3(modelauxNave * glm::vec4(0.0f, -1.0f, 0.0f, 0.0f)));
spotLights[2].SetFlash(navePos, naveDir);

//ALA DERECHA
modelNave = modelauxNave;
modelNave = glm::translate(modelNave, glm::vec3(-0.06f, -0.04f, -0.23f));
modelNave = glm::rotate(modelNave, -rotAla * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelNave));
AlaDerecha_M.RenderModel();

//ALA IZQUIERDA
modelNave = modelauxNave;
modelNave = glm::translate(modelNave, glm::vec3(-0.07f, -0.01f, 0.24f));
modelNave = glm::rotate(modelNave, rotAla * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelNave));
AlaIzquierda_M.RenderModel();

//HELICE IZQUIERDA
modelNave = modelauxNave;
modelNave = glm::translate(modelNave, glm::vec3(-0.37f, -0.29f, 0.36f));
modelNave = glm::rotate(modelNave, rothelice * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelNave));
Helice_M.RenderModel();

//HELICE DERECHA
modelNave = modelauxNave;
modelNave = glm::translate(modelNave, glm::vec3(-0.37f, -0.29f, -0.36f));
modelNave = glm::rotate(modelNave, -rothelice * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelNave));
Helice_M.RenderModel();

```



Extra)

Para la actividad extra, utilice el dado de 8 caras que hemos trabajado con anterioridad. A partir de una estructura que contiene las rotaciones específicas para que cada cara quede perpendicular al eje +Y, implemente una función que simula el lanzamiento del dado. Básicamente por tres segundos el dado rota aleatoriamente en (x,y,z) lo que simula un lanzamiento de dado. Y después de ese tiempo se selecciona una de las 8 configuraciones de las caras. Con la tecla “ESPACIO” se lanza el dado cada que el usuario lo desee.

```

////////////////////////////////////
//Rotaciones específicas para cada cara del dado D8 (en grados)
struct ConfigCara {
    float x;
    float z;
};

ConfigCara configsD8[8] = {
    { 35.26f,  45.0f }, // Cara 1
    { 35.26f, -45.0f }, // Cara 2
    {-35.26f, -45.0f }, // Cara 3
    {-35.26f,  45.0f }, // Cara 4
    { 144.74f,  45.0f }, // Cara 5
    { 144.74f, -45.0f }, // Cara 6
    {-144.74f, -45.0f }, // Cara 7
    {-144.74f,  45.0f } // Cara 8
};

```



```

//Logica animacion del dado de 8 caras
float tiempoActual = glfwGetTime();

if (mainWindow.lanzarDado && !estaAnimando) {
    estaAnimando = true;
    tiempoInicioAnim = tiempoActual;
}

if (estaAnimando) {
    float tiempoTranscurrido = tiempoActual - tiempoInicioAnim;

    if (tiempoTranscurrido < 2.0f) {
        //ROTACIÓN ALEATORIA (GIRO LOCO)
        anguloX += 500.0f * deltaTime;
        anguloY += 800.0f * deltaTime;
        anguloZ += 300.0f * deltaTime;
    }
    else {
        // SELECCION DE UNA CARA
        int resultado = mainWindow.caraD8;
        anguloX = configsD8[resultado].x;
        anguloZ = configsD8[resultado].z;
        anguloY = 0.0f; // El ajuste de cara ya se hace con X y Z

        estaAnimando = false;
        mainWindow.lanzarDado = false;
    }
}
}

```

Instancia dado 8 caras

```

//DADO 8 CARAS (OCTAEDRO)
glm::mat4 model(1.0f);
model = glm::translate(model, glm::vec3(30.0f, 1.0f, 0.0f)); |
model = glm::rotate(model, glm::radians(anguloX), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(anguloY), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::rotate(model, glm::radians(anguloZ), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(5.0f, 5.0f, 5.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
dadoTexture8.UseTexture();
meshList[7]->RenderMesh();

```




2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Me resulto complicado determinar cómo recorrer la pista ya que lo quería hacer con demasiados estados donde cada estado se movería de a poco el auto para determinar traslación y rotación (manera manual). Así que me ayude de una IA que me sugirió una solución que reduciría tiempo y código.

3.- Conclusión:

Determinar una correcta implementación de animación sugiere analizar lo que se requiere o desea generar para generar la mejor solución, ya que existen muchas formas de realizar una “cosa” y esto puede reducir el tiempo y código.